Golang Test File Naming Conventions
- Overview
- Test File Name Components
- File Name Prefix
- File Base Name
- File Number Component
- File Test Label
- File Extension
- Test File Location
- Test File Function Structure

Golang Test File Naming Conventions

Overview

Golang is another name for, and refers to, the **Go Programming Language**.

Test files contain code designed to test ***Golang*** source code.

Test files are typically placed in the same directory as source code files.

Test Files may contain one or more individual test functions. Individual test functions are usually designed to test a single source code function, method, type or operation.

Test file names consist of five components:

1. **File Name Prefix** (always set to "**zzzt_**")
2. **File Base Name** (Test Subject)
3. **File Number Component** (format "**_0000**")
4. **File Test Label** (always set to "**_test**")
5. **File Extension** (always set to "***.go***")

Test File Name Components

[FileNamePrefix] + [FileBaseName] + [FileNumberComponent] + [FileTestLabel] + [FileExtension]

Example:

> **zzzt_numberstrkernel_0020_test.go**

File Name Prefix

All test file names begin with a file name prefix of "**zzzt_**" .

This ensures that Test files will be sorted and listed separately from source code files stored in the same directory.

File Base Name

The **File Base Name** usually defines the source code function, method, type or operation which will be subjected to testing by the test functions contained in the subject test file.

File Number Component

The **File Number Component** consists of adding a trailing file number to the **File Base Name** using the format "**_0000**". The **File Number Component** is unique relative to the **File Base Name**.

File Numbers are typically increased in increments of ten to provide for future addition of new tests.

Examples:

```
zzzt_mathfloathelper_0010_test.go
zzzt_mathfloathelper_0020_test.go
zzzt_mathfloathelper_0030_test.go
zzzt_mathfloathelper_0040_test.go
zzzt_numberstrkernel_0010_test.go
zzzt_numberstrkernel_0020_test.go
zzzt_numberstrkernel_0030_test.go
zzzt_numstrhelper_0010_test.go
zzzt_numstrhelper_0020_test.go
```

If the number sequence becomes crowed, new files may be added using an extended **File Number Component** formatted as "**_0000_000_**". Example:

```
zzzt_mathfloathelper_0010_test.go
zzzt_mathfloathelper_0011_test.go
zzzt_mathfloathelper_0011_010_test.go # Extended File Number Component
zzzt_mathfloathelper_0012_test.go
```

File Test Label

The **File Test Label** component is added immediately after the **File Number Component**. The File Test Label is configured for all test files as "**_test**".

Example:

```
# All Files Labeled as 'test'
zzzt_mathfloathelper_0010_test.go
zzzt_mathfloathelper_0020_test.go
zzzt_mathfloathelper_0030_test.go
zzzt_mathfloathelper_0040_test.go
```

File Extension

All Test files are configured with a file extension of ***.go*** .

Test File Location

Test files are always located in the same directory as the source code files containing the source code subjected to testing.

Test File Function Structure

The following test file example contains only one test function. Test files may contain multiple test functions.

```
// Contents of Test File: zzzt_NaturalLogAGM_0010_test.go

func Test_Natural_Logarithm_AGM_0010(t *testing.T) {

  // Test Natural Logarithm of 5 calculated to 1004 digits of accuracy.
  // Expected Results (expectedResultStr) value was calculated using
  // Python's 'mpmath' library and function mp.ln(5).

  ePrefix := "Test_Natural_Logarithm_AGM_0010()"

  xValueStr := "5"

  expectedResultStr :=

 "1.609437912434100374600759333226187639525601354268517721912647891474178987707657764630133878093179610799966303021715562899724005229324676199633616617463705727552179637497183245653492856202341525057270155193600879777389725688193540712766154731221809527948521292821358059722567672285287240461589448178364671328673998424637759593189423843934353451050975054454194740501365987087867383213130572972040659485383838723662753876545562718161511659930915243207364911677863900675872585778766391583836823950425487956239484031001982697117490993741498480957621016911014378862403354321512723125734588461559787291980886570684020066599844472699973217681186517601220202978408109019647526699765068920658978913381571716207227774559770534320387787496829375361133800946764048330285002174774879708007143465616359897041258901337641524017005888598734948487787971054314178320201453062090615788790671598125167499041434172122036944700092685593759785492118961786432875876848322058355961996791316696509323388531585398982370985431463110008"

  bFloatHlpr := new(BigFloatHelper)

  var workingPrecision uint

  var sourceExpectedDecimalDigits, sourceInputDecimalDigits, sourceCalculatedDecimalDigits
int

  _, sourceInputDecimalDigits = bFloatHlpr.CountDigits(xValueStr, '.')
```

```
  _, sourceExpectedDecimalDigits = bFloatHlpr.CountDigits(expectedResultStr, '.')

  if sourceInputDecimalDigits > sourceExpectedDecimalDigits {
    sourceCalculatedDecimalDigits = sourceInputDecimalDigits
  } else {
    sourceCalculatedDecimalDigits = sourceExpectedDecimalDigits
  }

  sourceCalculatedDecimalDigits += 3

  workingPrecision =
bFloatHlpr.ComputeBigFloatPrecisionBits(uint(sourceCalculatedDecimalDigits), 1)

  xValue, isOk := new(big.Float).
    SetMode(big.AwayFromZero).
    SetPrec(workingPrecision).
    SetString(xValueStr)

  if !isOk {
    t.Errorf("%v\n"+
      "Error returned by:\n"+
      "xValue, isOk := new(big.Float)\n"+
      ".SetMode(big.AwayFromZero).SetPrec(workingPrecision).SetString(xValueStr)\n"+
      "xValueStr= '%v'\n"+
      "Working Precision Bits = '%v'\n"+
      "Error= 'SetString() FAILED!'\n\n", ePrefix, workingPrecision, xValueStr)
    return
  }

  actualResult, err := new(naturalLogAGM).lnAGMDirect(xValue, workingPrecision)

  if err != nil {

    t.Errorf("%v\n"+
      "Error returned by:\n"+
      "actualResult, err := new(naturalLogAGM).lnAGMDirect(xValue, workingPrecision)\n"+
      "xValue= '%v'\n"+
      "workingPrecision= '%v'\n"+
      "Error= '%v'\n\n",
      ePrefix,
      xValue.Text('f', sourceInputDecimalDigits),
      workingPrecision,
      err.Error())

    return
  }

  actualResultStr := actualResult.Text('f', sourceExpectedDecimalDigits)

  if expectedResultStr != actualResultStr {
    t.Errorf("%v\n"+
      "Error: Unexpected Result From AGM!\n"+
```

```
      "Because expectedResult != actualResultStr\n"+
      "Expected actualResultStr = '%v'\n"+
      "  Actual actualResultStr = '%v'\n\n",
      ePrefix, expectedResultStr, actualResultStr)
  }

  return
}
```